

Reinforcement Learning — Homework Assignment

Policy Methods with Continuous Action

Tal Fiskus
fiskust@biu.ac.il

Submission Deadline: 17/06/2026
Work can be done in pairs

Objective

The goal of this assignment is to implement one of the continuous action policy gradient algorithm we have seen in class in **PyTorch**.

As background for this assignment, you can read the PPO, DDPG, TD3 or SAC papers. (Links are in the Lemida course site)

Task Description

You should implement the agent from scratch in PyTorch and train it on **two** MuJoCo environments chosen from the following spreadsheet:

[Algorithm and Environment spreadsheet](#)

In that spreadsheet:

- Write your ids in the row of the algorithm you chose, based on open space.
- Choose two MuJoCo environments from the [gymnasium API](#).
- I suggest refreshing the page before selecting the algorithm so you won't override each other.

Implementation Requirements

1. Read the selected algorithm paper and implement the method in **PyTorch only**.
2. **Do not use any RL library.**
3. You could implement it in the same way it was implemented in the original paper or as we seen in class.
4. Use MuJoCo version v4
5. Train your agent on **two MuJoCo environments** and mark them in the spreadsheet.

6. You **can't use the exact hyperparameters** from the paper, play with them a little bit, and push yourself to the best.
7. **This is not a competition**, potentially you can all get the maximum grade, you only compete with yourself...

Training Requirements

1. You may train for a **limited number of 10 million training steps**. Here, a single call to `env.step(...)` is considered **one training step**.
2. During training, you must track performance over time:
 - Every **10,000 steps**, perform an evaluation.
 - In each evaluation, run one or more **evaluation episodes**.
 - Record the **total reward** obtained during evaluation.
3. In the literature, the standard practice is to evaluate over **100 episodes** and report the average score. You may use fewer evaluation episodes, in any case you **must clearly state in the report how many evaluation episodes you used, and how you evaluate the results**.
4. For the **final reported result**, you must train the agent 3 times per environment and report the **final rewards for these 3 different training runs**.

Report Requirements

Your final submission must include a **detailed PDF report**. The report should describe your code, implementation choices, and results.

At minimum, the report must include:

1. The **environments you chose**.
2. The **hyperparameters** you used.
3. A description of what you tried that **worked**.
4. A description of what you tried that **did not work** (for example: failed hyperparameter choices, etc.).
5. 3 graphs showing the **learning progress** for each training run: specifically, the evaluation rewards as a function of training time, with one point every **10k training steps**, until the training limit you chose.
6. The number of **evaluation episodes** used in each evaluation.
7. The number of **training steps** used in each training.
8. The **final reward**, reported as a list of **3 independent training runs**, and the average of these runs as a single scalar.

Submission Format

You must submit the following in a single zip file:

- A **PDF report**
- Your Python code.
- A video of the best trained agent for both environments

Collaboration Policy

You may work **individually or in pairs**.

Summary Checklist

- Read the chosen algorithm paper.
- Implement it in PyTorch from scratch.
- Use two MuJoCo environments.
- Evaluate every 10k steps.
- Plot evaluation reward versus training steps.
- Train 3 runs for each environment.
- Report the final result as an average over 3 runs.
- Submit a PDF report and a ZIP file with code.

Good Luck!